

الحصة الرابعة – 2025/9/14

1. الاختصارات في العمليات الحسابية
2. استقبال قيم عددية صحيحة وكسرية
3. التحويل بين أنواع الاعداد المختلفة Casting
4. كتابة برنامج

أولاً:  العمليات الحسابية المختصرة في C#

تعلمنا في الحصة السابقة عمليات حسابية بسيطة مثل الجمع الطرح وغيرها
وتعلمنا كيفية كتابة امر في عمليات حسابية بسيطة ومركبة
امثلة :

$x = y + 5 ;$

◆ ما هي العمليات المختصرة؟

العمليات المختصرة تُستخدم لتبسيط كتابة العمليات الحسابية مثل الجمع،
الطرح، الضرب، وغيرها على المتغيرات.

الوظيفة	الشكل الكامل المكافئ	الصيغة المختصرة
زيادة x بمقدار 1	$x = x + 1 ;$	$x += 1 ;$
إنقاص x بمقدار 2	$x = x - 2 ;$	$x -= 2 ;$
ضرب x في 3	$x = x * 3 ;$	$x *= 3 ;$
قسمة x على 2	$x = x / 2 ;$	$x /= 2 ;$
إيجاد باقي القسمة لـ x على 5	$x = x \% 5 ;$	$x \% = 5 ;$

أمثلة:

```
int a = 10;
```

```
a += 5; // الآن a = 15  a = a + 5
```

```
a *= 2; // الآن a = 30  a = a * 2
```

```
a -= 4; // الآن a = 26  a = a - 4
```

```
Console.WriteLine(a); // 26
```

ثانيًا: الفرق بين (x++) و (x--)

اضف للمتغير واحد $x = x + 1 \rightarrow x += 1 \rightarrow x++$

نقص من المتغير واحد $x = x - 1 \rightarrow x -= 1 \rightarrow x--$

تتبع مقطع البرنامج التالي واكتب ما هي التغييرات التي تحدث على المتغيرات

```
int x = 9, y = 2;
```

```
int w = x + y;
```

```
y += 10;
```

```
x++;
```

x	y	w	output
9	2	---	
9	2	11	
9	12	11	
10	12	11	

ثالثاً: التحويل بين الأنواع (Casting)

تعلمنا أنواع متغيرات مختلفة بلغة سي شارب مثل اعداد صحيحة `int` واعداد كسرية مثل `double` اذا اردنا حفظ متغيرات بين أنواع متغيرات مختلفة سنقع في أخطاء ومشاكل لهذا نلجأ الى التحويل `Casting` بين أنواع المتغيرات

♦ ما هو Casting ؟

هو عملية تحويل متغير من نوع إلى آخر مثل:

- (تحويل آمن) `double` إلى `int`
- (يُفقد الكسور) `int` إلى `double`

♦ أنواع التحويل:

النوع	هل يتم تلقائياً؟	هل يوجد فقدان بيانات؟	مثال
ضمني	نعم	لا	<code>int x = 5; double y = x;</code>
صریح	لا	نعم	<code>double z = 7.8; int w = (int)z;</code>

مثال :

```
double z = 7.81;
```

```
int w = (int)z;
```

```
int x = 34;
```

```
double y = x;
```

في المثال الأول ممنوع حفظ قيمة من متغير double الى متغير int مباشرة
هذا يتطلب casting اما العكس فهو صحيح وينفع بدون مشاكل

أمثلة: 

//تحويل ضمني:

```
int a = 10;
```

```
double b = a; // b = 10.0
```

```
double x = 9.7;
```

```
int y = (int)x; // y = 9
```

ملاحظة: التحويل الصريح يزيل الجزء العشري دون تقريب.

⚠ مثال مهم:

```
int a = 7;
```

```
int b = 2;
```

```
double result = (double)a / b;
```

result بدون التحويل: 3 — مع التحويل: 3.5

`Console.WriteLine(result);`

12 رابعًا: استقبال القيم من المستخدم `int` و `double`

* استخدام `Console.ReadLine()`

في C# ، جميع مدخلات المستخدم تُقرأ كنص (`string`) ، لذا يجب تحويلها إلى النوع المناسب.

استقبال عدد صحيح: (`int`) 📝

`Console.Write("أدخل عددًا صحيحًا");`

`int x = int.Parse(Console.ReadLine());`

`Console.WriteLine("العدد هو " + x);`

استقبال عدد عشري: (`double`) 📝

`Console.Write("أدخل عددًا عشريًا");`

`double y = double.Parse(Console.ReadLine());`

`Console.WriteLine("العدد هو " + y);`

ملاحظة: يجب استخدام النقطة للفاصلة العشرية مثال : 5.75

✓ مثال تطبيقي كامل:

`Console.Write("أدخل عددًا صحيحًا");`

`int a = int.Parse(Console.ReadLine());`

```
Console.Write("أدخل عددًا عشريًا");
```

```
double b = double.Parse(Console.ReadLine());
```

```
double sum = a + b;
```

```
Console.WriteLine("المجموع = " + sum);
```

ملخص سريع: 🧠

ملاحظات	الدالة أو الطريقة	الموضوع
يجب التأكد أن المُدخل رقم صحيح	<code>int.Parse()</code>	استقبال عدد صحيح
استخدم نقطة للفاصلة العشرية	<code>double.Parse()</code>	استقبال عدد عشري

بالنجاح للجميع